

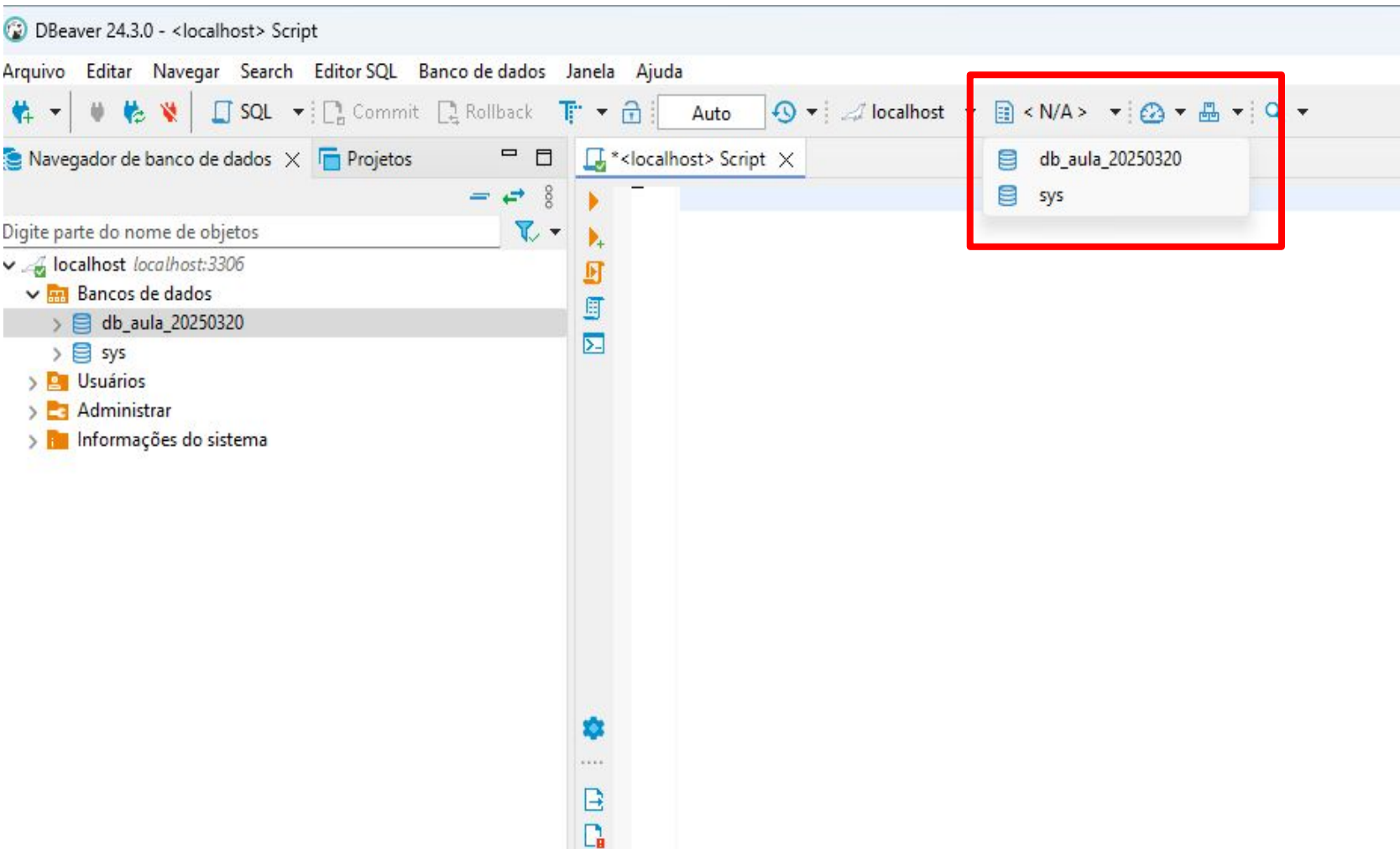
PROGRAMAÇÃO DE BANCO DE DADOS

Prof. Rafael Tápio

EXERCÍCIO

- Crie um banco de dados chamado **db_aula_20250320**.
- Use esse banco de dados para todos os exercícios da aula de hoje

```
CREATE DATABASE db_aula_20250320;  
USE db_aula_20250320;
```



USANDO RESTRIÇÕES

CONSTRAINTS (RESTRIÇÕES):

As constraints (restrições) são usadas para garantir a integridade dos dados em um banco de dados.

Elas definem regras que os dados devem seguir, como não permitir valores nulos ou garantir que uma chave estrangeira tenha um valor correspondente na tabela referenciada.

Podemos criar as constraints na no comando de criação das tabelas, mas às vezes, é necessário atualizar ou excluir essas restrições após a criação da tabela.

**PRIMARY
KEY**

CHAVE PRIMÁRIA:

A chave primária é uma restrição que garante que **cada valor na coluna ou conjunto de colunas seja único e não nulo**.

Ela é usada para identificar de forma exclusiva cada registro em uma tabela.

Normalmente, é criada na criação da tabela ou após, utilizando o comando **ALTER TABLE** para adicionar a restrição.

CHAVE PRIMÁRIA: CRIAÇÃO DA TABELA

Quando você cria uma tabela, você pode definir a chave primária diretamente na definição da coluna. Exemplo:

```
CREATE TABLE clientes (  
    id INT NOT NULL PRIMARY KEY,  
    nome VARCHAR(100)  
);
```

```
CREATE TABLE clientes (  
    id INT NOT NULL,  
    nome VARCHAR(100),  
    PRIMARY KEY (id)  
);
```

DBeaver 24.3.0 - clientes

Arquivo Editar Navegar Search Editor SQL Banco de dados Janela Ajuda

SQL Commit Rollback Auto localhost < N/A >

Navegador de banco de dados X Projetos

Propriedades Dados Diagrama ER localhost Bancos de dados

Digite parte do nome de objetos

- localhost localhost:3306
 - Bancos de dados
 - db_aula_20250320
 - Tabelas
 - clientes 16K

Nome da tabela: clientes ☐ Particionada

Motor: InnoDB

Auto-incremento: 0

Conjunto de caracteres:

Collation:

Descrição:

Colunas

Constraints

Chaves estrangeiras

Referências

Triggers

Índices

Partições

Statistics

DDL

Virtual

Nome	Coluna	Proprietário	Tipo	Verifica
PRIMARY	—	clientes	PRIMARY KEY	

CHAVE PRIMÁRIA: APÓS A CRIAÇÃO DA TABELA

Se a tabela já existe e você deseja adicionar uma chave primária, você pode fazer isso utilizando o comando **ALTER TABLE** para adicionar a restrição.

Exemplo:

```
ALTER TABLE clientes  
ADD CONSTRAINT pk_cliente PRIMARY KEY (id);
```

Aqui, estamos adicionando uma chave primária à coluna id da tabela clientes que já existe. A restrição é nomeada como **pk_cliente**.

CHAVE PRIMÁRIA: REMOVER

Se você quiser remover a chave primária de uma tabela, você pode usar o comando **ALTER TABLE** com **DROP PRIMARY KEY**.

```
ALTER TABLE clientes  
DROP PRIMARY KEY;
```

Isso remove a restrição de chave primária da tabela. Após esse comando, a coluna id deixará de ser a chave primária e poderá, por exemplo, ter valores duplicados ou nulos (se você permitir).

CHAVE PRIMÁRIA: CONSIDERAÇÕES

- Se a tabela já possui dados, a coluna usada como chave primária não pode ter valores nulos ou duplicados. Caso contrário, o comando ALTER TABLE retornará um erro.
- Se você estiver criando uma chave primária em uma coluna existente, e a coluna ainda não estiver marcada como NOT NULL, você precisará primeiro adicionar a restrição NOT NULL, como neste exemplo:

```
ALTER TABLE clientes  
MODIFY id INT NOT NULL;
```

```
ALTER TABLE clientes  
ADD CONSTRAINT pk_cliente PRIMARY KEY (id);
```

UNIQUE

UNIQUE

A restrição **UNIQUE** garante que todos os valores em uma coluna (ou conjunto de colunas) sejam únicos dentro da tabela, ou seja, não pode haver dois registros com o mesmo valor. Isso é semelhante à chave primária, mas há uma diferença fundamental:

- A chave primária não permite valores nulos e deve ser única por definição.
- A restrição **UNIQUE**, por outro lado, permite valores nulos. Ou seja, você pode ter várias linhas com valores nulos em uma coluna definida como **UNIQUE**.

UNIQUE

Quando você cria uma tabela, você pode adicionar uma restrição **UNIQUE** a uma ou mais colunas para garantir que seus valores sejam exclusivos, sem precisar de uma chave primária.

```
CREATE TABLE clientes (  
    id INT NOT NULL PRIMARY KEY,  
    email VARCHAR(100) UNIQUE,  
    nome VARCHAR(100)  
);
```

Aqui, a coluna email é marcada com a restrição UNIQUE, o que significa que não podem existir dois registros com o mesmo email.

Porém, ao contrário da chave primária, você poderia ter valores nulos na coluna email, caso isso fosse necessário (se a coluna email fosse opcional).

UNIQUE

Caso a tabela já exista e você deseje adicionar a restrição UNIQUE a uma coluna já criada, você pode usar o comando **ALTER TABLE**:

```
ALTER TABLE clientes  
ADD CONSTRAINT uq_email UNIQUE (email);
```

Aqui, estamos criando a restrição **uq_email** para garantir que a coluna email tenha valores únicos.

UNIQUE

Para remover uma restrição **UNIQUE**, você deve saber o nome da restrição. Se você a nomeou ao criar a tabela ou ao adicionar a restrição, como `uq_email`, você pode usar o nome da restrição diretamente. Caso contrário, o banco de dados pode gerar um nome automaticamente.

```
ALTER TABLE clientes  
DROP CONSTRAINT uq_email;
```

UNIQUE

Podemos utilizar mais de 1 campo no **UNIQUE**.

```
CREATE TABLE funcionarios (  
    id INT NOT NULL PRIMARY KEY,  
    nome VARCHAR(100),  
    email VARCHAR(100),  
    telefone VARCHAR(20),  
    CONSTRAINT uq_email_telefone UNIQUE (email, telefone)  
);
```

CHECK

CHECK

A restrição **CHECK** é usada para garantir que os dados inseridos em uma coluna satisfaçam uma condição específica.

Ela permite que você defina regras de validade para os dados, como garantir que um valor esteja dentro de um intervalo, que seja maior que outro valor, ou que tenha algum outro tipo de condição lógica.

CHECK

Vamos imaginar que você tenha uma tabela de funcionários e queira garantir que o salário dos funcionários seja maior que 1000. A restrição CHECK seria usada para impor essa regra.

```
CREATE TABLE funcionarios (  
    id INT NOT NULL PRIMARY KEY,  
    nome VARCHAR(100),  
    salario DECIMAL(10, 2),  
    CONSTRAINT chk_salario CHECK (salario > 1000)  
);
```

Neste exemplo:

- A restrição **CHECK** é aplicada à coluna salario, garantindo que o salário de cada funcionário seja sempre maior que 1000.
- A restrição é nomeada como **chk_salario** para facilitar a identificação.

CHECK

Aqui está um exemplo de como definir uma restrição CHECK diretamente no campo, sem precisar nomear a restrição.

```
CREATE TABLE funcionarios (  
    id INT NOT NULL PRIMARY KEY,  
    nome VARCHAR(100),  
    salario DECIMAL(10, 2) CHECK (salario > 1000)  
);
```

CHECK

Caso você tenha uma tabela já criada e queira adicionar uma restrição CHECK a uma coluna, você pode usar o comando ALTER TABLE.

Exemplo:

```
ALTER TABLE funcionarios  
ADD CONSTRAINT chk_salario CHECK (salario > 1000);
```

Isso adiciona a restrição CHECK à tabela existente, garantindo que qualquer valor inserido na coluna salario seja maior que 1000.

CHECK

Para remover a restrição CHECK de uma tabela, você deve usar o comando ALTER TABLE com DROP CONSTRAINT. Assim como no caso de UNIQUE, a restrição CHECK possui um nome, que pode ser usado para referenciá-la ao removê-la.

Exemplo:

```
ALTER TABLE funcionarios  
DROP CONSTRAINT chk_salario;
```

Aqui, estamos removendo a restrição **chk_salario** que garante que o salário seja maior que 1000.

Nota: Ao remover a restrição CHECK, o banco de dados deixa de validar automaticamente as condições para a coluna afetada.

CHECK

Podemos utilizar mais de 1 campo no CHECK, para compor a regra. Exemplo:

```
CREATE TABLE funcionarios (  
    id INT NOT NULL PRIMARY KEY,  
    nome VARCHAR(100),  
    experiencia_anos INT,  
    salario DECIMAL(10, 2),  
    CONSTRAINT chk_salario_experiencia CHECK  
        (salario > experiencia_anos * 1000)  
);
```

CHECK

Exemplo2:

```
CREATE TABLE produtos (  
    id INT NOT NULL PRIMARY KEY,  
    nome VARCHAR(100),  
    preco_custo DECIMAL(10, 2),  
    preco_venda DECIMAL(10, 2),  
    CONSTRAINT chk_preco CHECK  
        (preco_venda >= preco_custo)  
);
```

EXERCÍCIOS

EXERCÍCIO 1 - Passo 1: Criar a tabela

Crie uma tabela chamada empregados para armazenar informações sobre os empregados de uma empresa. Inicialmente, crie a tabela **sem constraints**.

- **id_empregado** (número inteiro, não nulo) – Identificador único do empregado.
- **nome** (texto, até 100 caracteres) – Nome completo do empregado.
- **email** (texto, até 150 caracteres) – Endereço de e-mail do empregado.
- **data_admissao** (data) – Data de admissão do empregado.
- **salario** (decimal, até 2 casas decimais) – Salário do empregado.
- **departamento** (texto, até 50 caracteres) – Departamento no qual o empregado trabalha.

EXERCÍCIO 1 - Passo 2: Adicionar as constraints

Após a criação da tabela, adicione as seguintes constraints utilizando o comando ALTER TABLE:

- **PRIMARY KEY:** Defina a coluna id_empregado como chave primária e nomeie a constraint como pk_empregado.
- **UNIQUE:** Defina a coluna email como única (não pode haver dois empregados com o mesmo e-mail) e nomeie a constraint como uq_email.
- **CHECK 1:** Adicione uma restrição CHECK para garantir que o salário seja maior que 1000. Nomeie a constraint como chk_salario_minimo.
- **CHECK 2:** Adicione uma restrição CHECK para garantir que o departamento não seja vazio. Nomeie essa constraint como chk_departamento_nao_vazio.

EXERCÍCIO 1 - Passo 3: Excluir Constraints

Passo 3: Excluir uma das Constraints

Remova a constraint **chk_departamento_nao_vazio**.

EXERCÍCIO 1 - Passo 4: Inserir Dados

Passo 4: Inserir Dados na Tabela

Proponha as inserções/tentativas abaixo:

- 3 INSERTs que terão sucesso.
- 1 INSERT com erro devido ao CHECK do salário.
- 1 INSERT com erro devido à CONSTRAINT de UNIQUE.
- 1 INSERT com erro devido à PRIMARY KEY.
- 1 INSERT com sucesso, permitindo o departamento vazio (após a remoção da constraint CHECK).

*<localhost> Script

empregados X

Propriedades Dados Diagrama ER

localhost

Nome da tabela: empregados ☐ Particionada

Motor: InnoDB

Auto-incremento: 0

Conjunto de caracteres:

Collation:

Descrição:

+ Colunas

Constraints

Chaves estrangeiras

Referências

Triggers

Índices

Partições

Statistics

DDL

Virtual

Nome	Coluna	Proprietário	Tipo	Verificar expressão
> PRIMARY	—	empregados	PRIMARY KEY	
> uq_email	—	empregados	UNIQUE KEY	
> chk_salario_minimo	—	empregados	CHECK	`salario` > 1000

EXERCÍCIO 2:

Crie uma tabela para armazenar informações sobre dados diversos (à sua escolha: produto, cliente, fornecedor, cidades, sistemas, estoque, outros..). A tabela deve ter pelo menos **10 campos**, e as seguintes restrições:

- Uma **chave primária** composta de **dois campos**.
- Uma restrição **UNIQUE** em uma coluna diferente da chave primária.
- Pelo menos **duas** restrições **CHECK**, para garantir que os dados atendam a critérios específicos .
- Obrigatoriamente, utilize os tipos de dados **VARCHAR**, **INT**, **DECIMAL(10,2)**, **DATE**, e **BOOLEAN**.

Após a criação da tabela, insira 10 registros válidos.

EXERCÍCIO 3:

Com a tabela cujo script está no próximo slide:

- Adicione a chave primária (PK), que deve ser composta por dois campos.
- Adicione uma constraint UNIQUE para garantir que o nome do produto seja único.
- Adicione duas constraints CHECK:
 - A primeira deve garantir que a quantidade do produto não seja negativa.
 - A segunda deve garantir que a data de validade do produto seja maior que a data de fabricação.

EXERCÍCIO 3:

```
CREATE TABLE produtos (  
    id_produto INT,  
    codigo_produto VARCHAR(20),  
    nome_produto VARCHAR(100),  
    descricao_produto VARCHAR(255),  
    quantidade_estoque INT,  
    preco_produto DECIMAL(10, 2),  
    data_fabricacao DATE,  
    data_validade DATE,  
    ativo BOOLEAN  
);
```